Code Explanation Line by Line (Easy + Detailed)

import numpy as np

Explanation:

- Imports NumPy library.
- Used for:
    - Arrays
    - Matrix calculations
    - Mathematical operations

Sigmoid Function

def sigmoid(x):

Explanation:

- Defines sigmoid activation function.

---

return 1 / (1 + np.exp(-x))

Explanation:

Sigmoid formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Purpose:

- Converts values into range:

0 to 1

Sigmoid Derivative

def sigmoid_derivative(x):

Explanation:

- Defines derivative of sigmoid function.
- Used during backpropagation.

---

return x * (1 - x)

Explanation:

Derivative formula:

$$\sigma'(x) = x(1 - x)$$

Used for:

- Gradient calculation
- Weight updating

XOR Dataset

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

Explanation:

- Input features for XOR gate.

Inputs:

[0,0]
[0,1]
[1,0]
[1,1]

---

y = np.array([[0], [1], [1], [0]])

Explanation:

- Expected outputs of XOR gate.

Random Seed

np.random.seed(1)

Explanation:

- Ensures same random values every run.
- Makes output reproducible.

Define Neurons

input_neurons, hidden_neurons, output_neurons = 2, 4, 1

Explanation:

Neural network structure:

- 2 input neurons
- 4 hidden neurons
- 1 output neuron

Initialize Weights

weights_input_hidden = np.random.rand(input_neurons, hidden_neurons)

Explanation:

- Random weights between:
    - Input layer
    - Hidden layer

Shape:

2 × 4

---

weights_hidden_output = np.random.rand(hidden_neurons, output_neurons)

Explanation:

- Random weights between:
    - Hidden layer
    - Output layer

Shape:

4 × 1

Initialize Biases

bias_hidden = np.random.rand(1, hidden_neurons)

Explanation:

- Random biases for hidden layer.

---

bias_output = np.random.rand(1, output_neurons)

Explanation:

- Random bias for output layer.

Training Parameters

epochs = 10000

Explanation:

- Total training iterations.

---

learning_rate = 0.1

Explanation:

- Controls weight update size.

Small learning rate:

- Slow learning

Large learning rate:

- Unstable learning

Training Loop

for epoch in range(epochs):

Explanation:

- Runs training repeatedly for 10000 epochs.

Forward Propagation

hidden_input = np.dot(X, weights_input_hidden) + bias_hidden

Explanation:

Calculates hidden layer input.

Formula:

$$Z = W \cdot X + b$$

---

hidden_output = sigmoid(hidden_input)

Explanation:

- Applies sigmoid activation function on hidden layer.

---

final_input = np.dot(hidden_output, weights_hidden_output) + bias_output

Explanation:

- Calculates output layer input.

---

final_output = sigmoid(final_input)

Explanation:

- Final prediction generated.

Error Calculation

error = y - final_output

Explanation:

- Calculates prediction error.

Difference between:

- Actual output
- Predicted output

Output Layer Gradient

d_output = error * sigmoid_derivative(final_output)

Explanation:

- Computes output layer gradients.

Used for:

- Backpropagation
- Weight updating

Hidden Layer Error

error_hidden = d_output.dot(weights_hidden_output.T)

Explanation:

- Sends error backward to hidden layer.

---

d_hidden = error_hidden * sigmoid_derivative(hidden_output)

Explanation:

- Calculates hidden layer gradients.

Update Weights

weights_hidden_output += hidden_output.T.dot(d_output) * learning_rate

Explanation:

- Updates weights between hidden and output layer.

---

weights_input_hidden += X.T.dot(d_hidden) * learning_rate

Explanation:

- Updates weights between input and hidden layer.

Update Biases

bias_output += np.sum(d_output, axis=0, keepdims=True) * learning_rate

Explanation:

- Updates output bias.

---

bias_hidden += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

Explanation:

- Updates hidden bias.

Print Loss

if epoch % 1000 == 0:

Explanation:

- Prints loss every 1000 epochs.

---

loss = np.mean(np.abs(error))

Explanation:

- Calculates average error.

---

print(f"Epoch {epoch}, Loss: {loss:.4f}")

Explanation:

- Displays training progress.

Example:

Epoch 1000, Loss: 0.1200

Testing Phase

print("\nFinal Predictions:")

Explanation:

- Prints final prediction heading.

---

for i in range(len(X)):

Explanation:

- Tests all XOR inputs.

---

hidden_layer = sigmoid(np.dot(X[i], weights_input_hidden) + bias_hidden)

Explanation:

- Computes hidden layer output.

---

output = sigmoid(np.dot(hidden_layer, weights_hidden_output) + bias_output)

Explanation:

- Computes final prediction.

---

print(f"Input: {X[i]} => Predicted: {output[0][0]:.4f}")

Explanation:

- Displays predicted output.

Example:

Input: [1 0] => Predicted: 0.9876

What Happens Internally in This Code? (Detailed Explanation)

This program implements an Artificial Neural Network (ANN) using Forward Propagation and Backpropagation to solve the XOR problem. First, NumPy library is imported for mathematical and matrix operations. Then sigmoid activation function and its derivative are defined. Sigmoid converts outputs into values between 0 and 1, while its derivative helps during backpropagation for updating weights.

The XOR dataset is created with four input combinations and corresponding outputs. After that, the neural network structure is initialized with 2 input neurons, 4 hidden neurons, and 1 output neuron. Random weights and biases are generated between input-hidden layer and hidden-output layer. Training parameters such as epochs and learning rate are also defined.

During Forward Propagation, inputs move from input layer to hidden layer and finally to output layer. At each layer, weighted sum and bias are calculated, and sigmoid activation function is applied. The network then generates predicted outputs. After prediction, Backpropagation starts. The error between actual output and predicted output is calculated. This error is propagated backward through the network, and gradients are calculated using sigmoid derivative.

Finally, weights and biases are updated repeatedly to reduce prediction error and improve model accuracy. This entire process runs for 10000 epochs. As training progresses, loss gradually decreases, showing that the model is learning successfully. In the testing phase, the trained ANN predicts outputs for all XOR inputs, demonstrating how neural networks solve non-linear problems using hidden layers and backpropagation learning.

1. What is Artificial Neural Network (ANN)?

ANN is a computational model inspired by the human brain used for learning patterns and solving machine learning problems.

---

2. What is Forward Propagation?

Forward propagation is the process in which input data moves from input layer to output layer to generate prediction.

---

3. What is Backpropagation?

Backpropagation is the process of sending error backward through the network to update weights and reduce prediction error.

---

4. Why is backpropagation important?

It helps the neural network learn by minimizing error and improving prediction accuracy.

---

5. What is the role of weights in ANN?

Weights determine the importance of input features in prediction.

---

6. What is bias in neural network?

Bias helps shift activation function and improves model learning capability.

---

7. Which activation function is used in this program?

Sigmoid activation function is used.

Formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

---

8. What is the output range of sigmoid function?

The output range is:

0 to 1

---

9. Why is sigmoid function used?

Because it converts outputs into probabilities and is useful for binary classification.

---

10. What is XOR problem?

XOR is a logical operation where output is 1 when inputs are different and 0 when inputs are same.

Input	Output
0 0	0
0 1	1
1 0	1
1 1	0

---

11. Why is XOR called a non-linear problem?

Because XOR data cannot be separated using a straight line.

---

12. Why are hidden layers needed in XOR problem?

Hidden layers help ANN learn non-linear relationships.

---

13. What is learning rate?

Learning rate controls how much weights are updated during training.

---

14. What happens if learning rate is too high?

Training may become unstable and model may not learn properly.

---

15. What happens if learning rate is too low?

Training becomes very slow.

---

16. What is epoch?

One complete training cycle of the entire dataset is called an epoch.

---

17. How many epochs are used in this program?

10000 epochs

---

18. What is loss function?

Loss function measures prediction error of the model.

---

19. Why is NumPy used in this program?

NumPy is used for:

- Arrays
- Matrix operations
- Mathematical calculations

---

20. What does np.dot() do?

It performs matrix multiplication.

---

21. What is the purpose of random initialization of weights?

Random weights help neural network start learning from different values.

---

22. What is overfitting?

Overfitting occurs when a model memorizes training data and performs poorly on new data.

---

23. What is underfitting?

Underfitting occurs when model cannot learn patterns properly.

---

24. What is hidden layer?

Hidden layer is the intermediate layer between input and output layer where feature learning occurs.

---

25. How many hidden neurons are used in this code?

4 hidden neurons

---

26. What is the purpose of activation function?

Activation function introduces non-linearity into neural network.

---

27. What is gradient in backpropagation?

Gradient shows direction and amount of weight update needed to reduce error.

---

28. What is derivative of sigmoid function?

$$\sigma'(x) = x(1 - x)$$

---

29. What are the applications of ANN?

Applications include:

- Image recognition
- Speech recognition
- Robotics
- NLP
- Medical diagnosis

---

30. What is the main goal of this practical?

To implement ANN training using Forward Propagation and Backpropagation for solving XOR classification problem.